Database Technology Justification

Why PostgreSQL Was Chosen Over MySQL
Barcode-Based Student Verification System

This document explains the technical reasoning behind selecting **PostgreSQL** as the database engine for the Barcode-Based Student Verification System, rather than MySQL. The decision was driven by security, data integrity, concurrency, and platform considerations specific to this system — not by cost, since both databases are free.

1. The Platform Choice: Supabase Is Built on PostgreSQL

The system uses **Supabase** as its backend platform. Supabase provides authentication, role-based access control, realtime updates, and a hosted database in a single managed service — and it is built entirely on PostgreSQL. Supabase chose PostgreSQL as its foundation precisely because of the capabilities described in this document; MySQL could not power such a platform. By choosing PostgreSQL, the project gained a production-grade backend (authentication, security, APIs, realtime updates) without months of additional server development and ongoing maintenance.

2. Row Level Security — Security Enforced Inside the Database

This is the single most important technical reason for a system that handles student data. PostgreSQL supports **Row Level Security (RLS)**: access rules written into the database itself that control which rows each user may read or modify. For example:

- A gate guard can be restricted to **reading** student records and **writing** scan logs only.
- Only administrators can modify the watchlist, gates, or user accounts.
- Sensitive records (incidents, reports) can be hidden from roles that have no business viewing them.

Even if a bug existed in the application code, the database itself would refuse unauthorized access. MySQL has **no equivalent feature** — with MySQL, all such security would live only in application code, which is a far weaker guarantee for a school security system.

3. Data Integrity — The Database Refuses Bad Data

Every entry/exit log is an audit record and must be trustworthy. PostgreSQL is strict: invalid dates, out-of-range values, and violated constraints are **rejected with an error**. MySQL, by contrast, has a long history of silently “helping” — truncating strings that are too long, accepting invalid dates such as 0000-00-00, and coercing bad values, depending on configuration. For attendance and incident records that may need to be defended later (“was this student on campus at 2:14 PM?”), silent data corruption is unacceptable.

4. Concurrency — Many Gates Scanning at Once

PostgreSQL’s multi-version concurrency control (MVCC) means readers never block writers and writers never block readers. Multiple gates can scan barcodes and write logs simultaneously while

dashboards and reports read the same tables, with no locking contention. PostgreSQL also provides full ACID transaction guarantees on every table by default, whereas MySQL only achieves this with the InnoDB engine and shows weaker consistency behavior under heavy concurrent load.

5. Better Data Types for This Exact Domain

- **timestamptz** — timezone-aware timestamps, so entry/exit times are always unambiguous. MySQL's timezone handling is notoriously error-prone.
- **Native UUID** — safe unique identifiers for students, visitors, and scan events, generated inside the database itself.
- **JSONB** — flexible structured data (incident details, notification payloads) that can still be indexed and queried efficiently. MySQL's JSON support is significantly less capable.
- **Enums, arrays, and check constraints** — statuses such as gate states or watchlist reasons are enforced by the database, not merely by convention.

6. Reporting Power

The system includes dashboards and reports: attendance summaries, peak entry times, and incident histories. PostgreSQL's window functions, common table expressions (CTEs), FILTER clauses, and partial indexes make these queries both easier to write and faster to run. MySQL has narrowed some of this gap in recent versions, but PostgreSQL remains ahead, and its query planning is more predictable as data volume grows.

7. Realtime Capability

PostgreSQL has built-in change notification (LISTEN/NOTIFY and logical replication), which is exactly what powers Supabase Realtime. This is how the dashboard can display a scan the moment it happens at a gate, without polling. The capability is architecturally native to PostgreSQL; with MySQL it would require bolting on external tooling.

8. Licensing and Governance

PostgreSQL is truly open source under a permissive license and is developed by an independent community — no single company controls it. MySQL is owned by Oracle, carries a dual GPL/commercial license, and has a history of features being reserved for the paid edition. For a long-lived institutional system, PostgreSQL carries zero vendor-lock-in and zero licensing risk.

9. Side-by-Side Summary

Criterion	PostgreSQL	MySQL
Row Level Security	Built-in	Not available
Strict data validation	Always enforced	Configuration-dependent; historically silent coercion
ACID transactions	Default on all tables	InnoDB engine only

Timezone-aware timestamps	Native (timestamptz)	Error-prone handling
Native UUID type	Yes	No (stored as strings/binary)
JSON support	JSONB — indexed, queryable	Limited JSON functions
Realtime change feeds	LISTEN/NOTIFY, logical replication (powers Supabase)	Requires external tooling
Advanced reporting SQL	Full window functions, CTEs, partial indexes	Partial support, gap remains
Ownership / license	Independent community, permissive license	Oracle-owned, dual license
Cost	Free	Free (community edition)

10. Conclusion

Both databases are free, and at this system's scale raw performance would be similar. The decision was therefore about **security, data integrity, and platform capability** — not speed or cost.

> *PostgreSQL provides security enforced inside the database (Row Level Security) for protecting student records, strict integrity for audit-grade entry/exit logs, robust handling of simultaneous scans from multiple gates, and native realtime updates for the live dashboard. It is also the foundation of Supabase, which supplied authentication and a secure API out of the box instead of months of extra development. Achieving the same with MySQL would have required rebuilding all of that in application code, with weaker guarantees.*